

CHƯƠNG 4: PHÁT HIỆN VÀ MÔ TẢ ĐẶC TRƯNG ẢNH

Tiết 8/9: So khớp đặc trưng (Feature Matching)

Môn học:	Thị giác máy tính (INFO3111)
----------	------------------------------

Lớp:	IT23M
------	-------

Giảng viên:	Trần Thành Thắng
-------------	------------------

Mục tiêu bài học (Tiết 8)

KIẾN THỨC (Lý thuyết)



Bản chất So khớp: Thiết lập các cặp tương ứng (correspondences) giữa các không gian ảnh.



Động cơ Cốt lõi: Phân biệt cơ chế hoạt động giữa Brute-Force (vét cạn) và FLANN (xấp xỉ).



Độ đo Toán học: Lựa chọn đúng khoảng cách L2 (Euclidean) hoặc Hamming cho từng loại vector mô tả.

KỸ NĂNG (Thực hành OpenCV)



Triển khai BFMatcher: Sử dụng thành thạo lớp `cv2.BFMatcher` trong bài toán tìm kiếm vật thể.



Bộ lọc Lowe: Cài đặt kỹ thuật KNN và Ratio Test để loại bỏ triệt để các cặp khớp sai (False Matches).

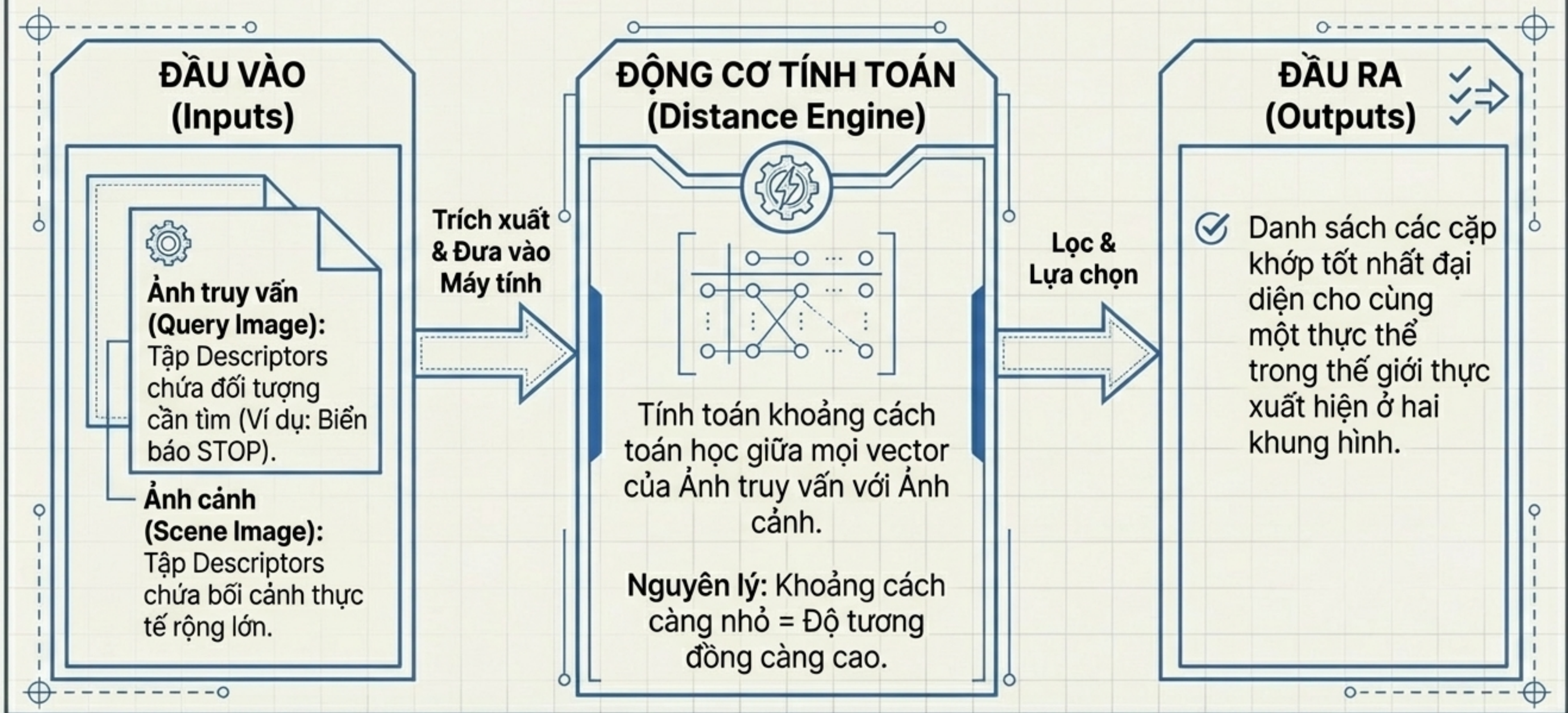
So khớp đặc trưng (Feature Matching) là gì?

Quá trình thiết lập mối liên hệ giữa các điểm đặc trưng (keypoints) từ các ảnh khác nhau dựa trên sự tương đồng của các vector mô tả (descriptors).



Tầm quan trọng: Nền tảng cốt lõi cho mọi ứng dụng thị giác máy tính bậc cao: Tìm kiếm vật thể, Ghép ảnh Panorama, Theo dõi chuyển động (Tracking), và Tái tạo 3D.

Quy trình chuẩn của một Bộ so khớp (Matcher Pipeline)

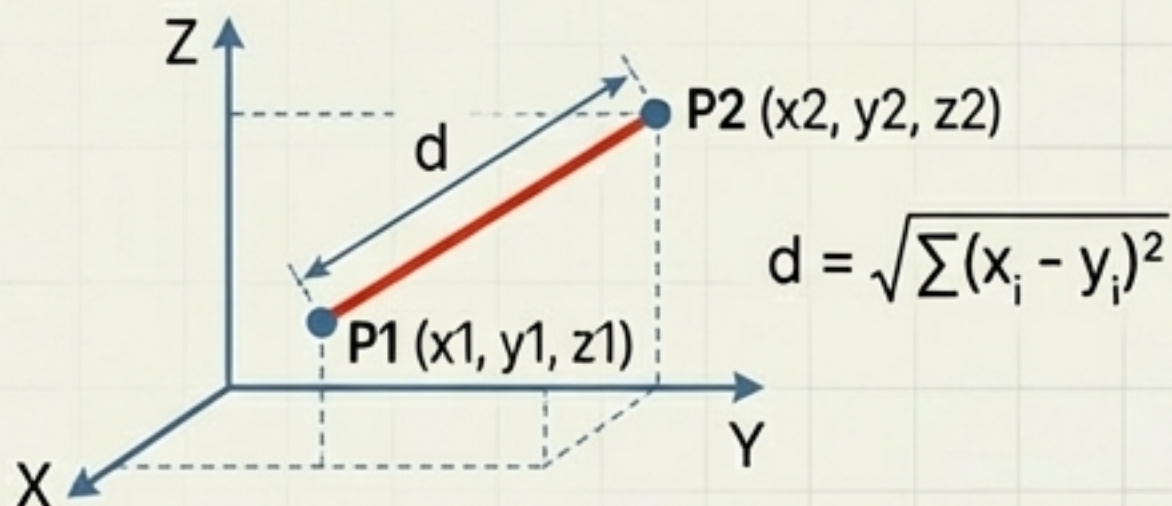


Bộ cơ sở 1: Brute-Force Matcher & Độ đo khoảng cách

Cơ chế **vét cạn (Exhaustive Search)**: Với mỗi descriptor trong Ảnh truy vấn, thuật toán tính toán khoảng cách đến tất cả các descriptor trong Ảnh cảnh để tìm ra một kết quả tốt nhất. Trực diện, chính xác 100%, nhưng tốn kém tài nguyên.

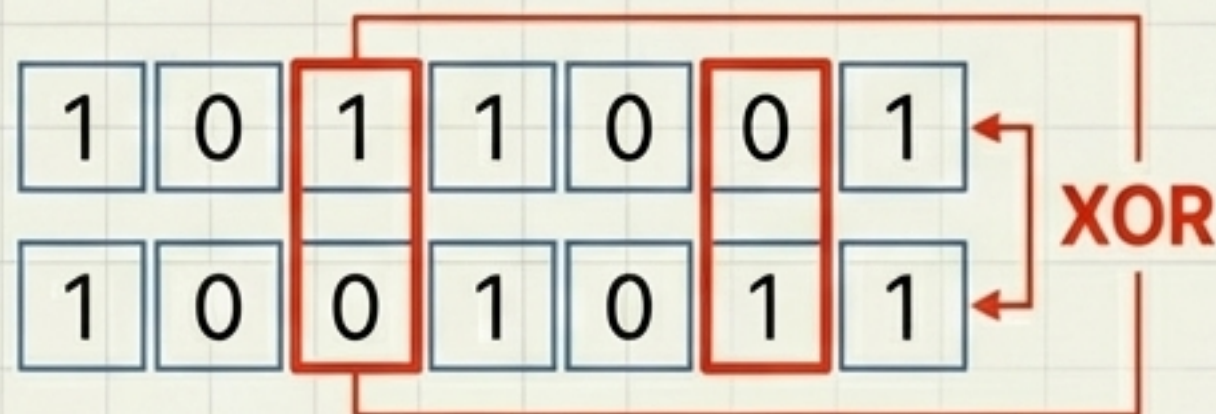
```
cv2.BFMatcher(normType, crossCheck)
```

Khoảng cách L2 (Euclidean)



- **Đặc điểm:** Tính khoảng cách hình học trong không gian đa chiều liên tục.
- **Áp dụng:** Các bộ mô tả có giá trị số thực (SIFT, SURF).

Khoảng cách Hamming



- **Đặc điểm:** So sánh bằng phép toán XOR (đếm số bit khác nhau).
- **Áp dụng:** Các bộ mô tả nhị phân (ORB, BRIEF) để tối ưu tốc độ cực đại.

Bộ cơ sở 2: FLANN

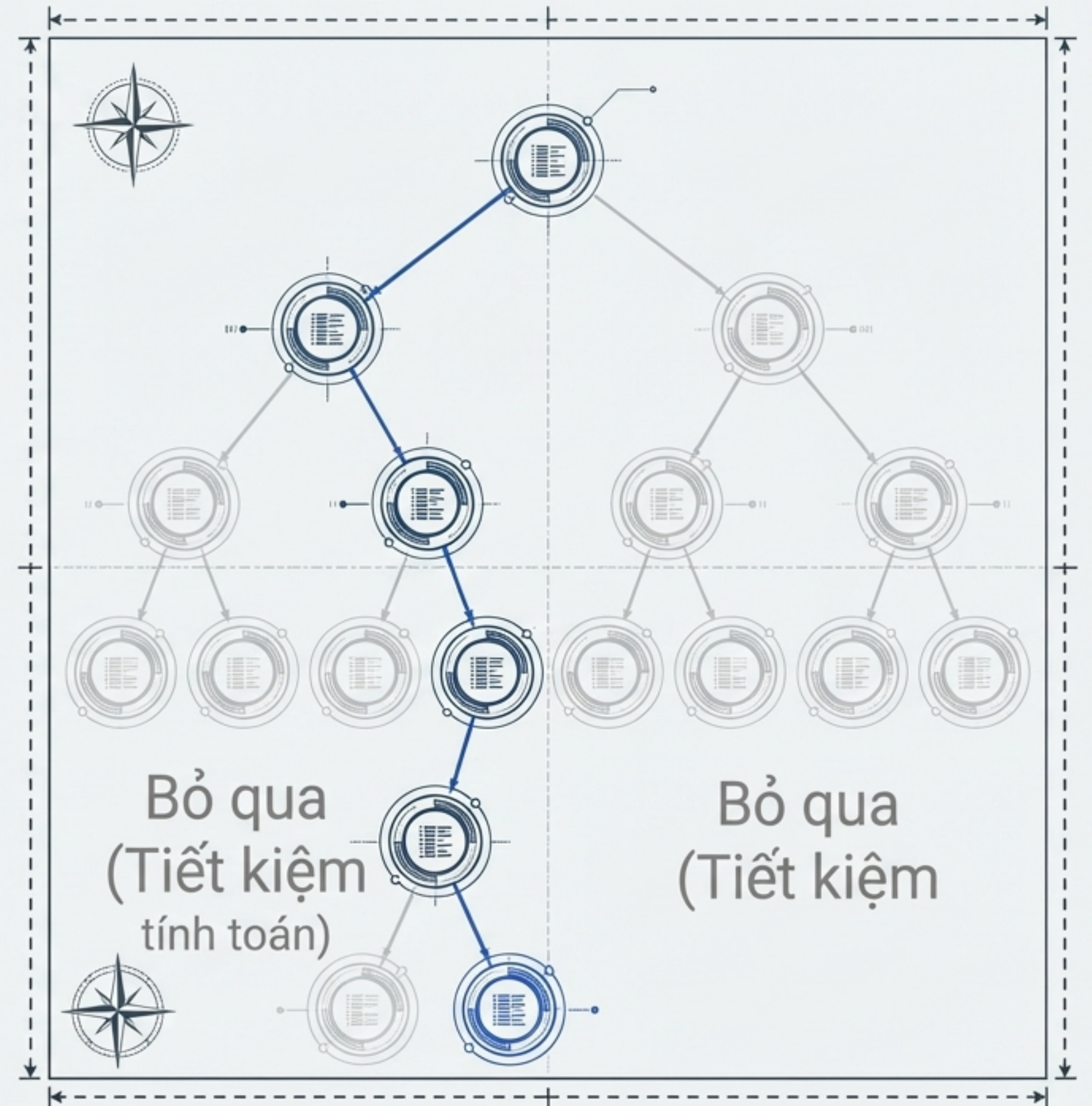
Matcher (Tìm kiếm Xấp xỉ)

Fast Library for Approximate Nearest Neighbors

Tại sao cần FLANN? Khi tập dữ liệu lên tới hàng triệu điểm (Scale lớn), thuật toán Brute-Force trở nên quá chậm. FLANN giải quyết bài toán bằng cách hy sinh một lượng nhỏ độ chính xác để đổi lấy tốc độ vượt trội.

Cơ chế cốt lõi: Cấu trúc Cây K-D (K-D Trees)

- Thay vì quét toàn bộ dữ liệu, FLANN phân chia không gian đặc trưng thành các nhánh cây.
- Khi truy vấn, thuật toán chỉ "leo" xuống các nhánh có khả năng chứa láng giềng gần nhất, bỏ qua các cụm không liên quan.
- Hiệu năng: Tốc độ truy vấn có thể nhanh hơn gấp 10 lần.



Ma trận Tổng hợp: Lựa chọn Động cơ So khớp

Tiêu chí	Brute-Force (BF Matcher)	FLANN Matcher
Cơ chế tìm kiếm	Vét cạn (Exhaustive)	Xấp xỉ láng giềng (ANN / KD-Tree)
Độ chính xác	Tuyệt đối (100%)	Tương đối (Xấp xỉ rất gần)
Tốc độ xử lý	Chậm (Độ phức tạp $O(N \times M)$)	Cực nhanh (Độ phức tạp $O(\log N)$)
Phù hợp nhất cho	Ứng dụng nhỏ, cần chính xác tuyệt đối, bộ mô tả nhị phân (ORB)	Dữ liệu khổng lồ, ứng dụng thời gian thực (Real-time), SIFT/SURF

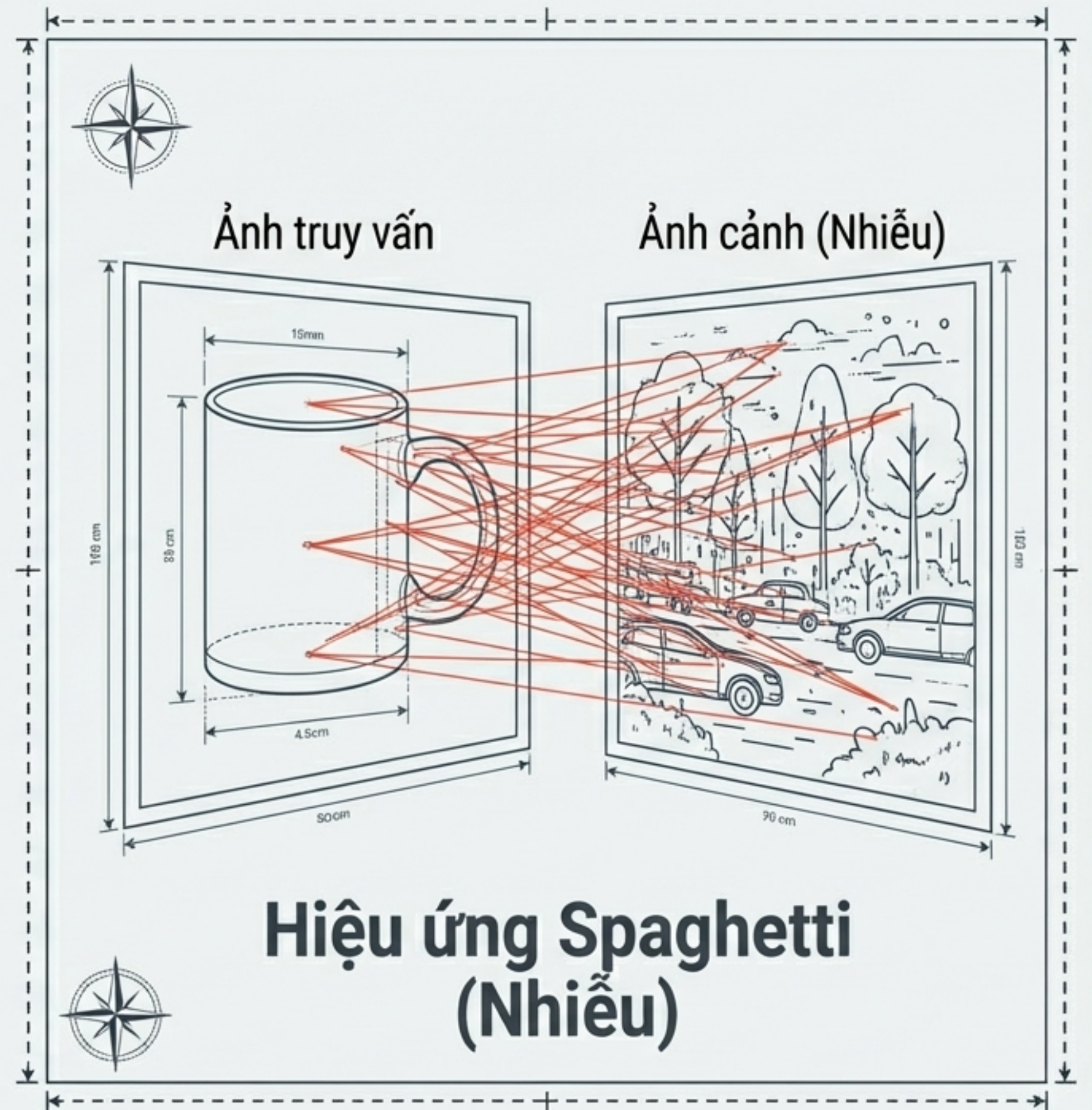
Lời khuyên thực chiến: Trong OpenCV, **cv2.FlannBasedMatcher** có khả năng tự động xác định thuật toán (KD-Tree hay LSH) tối ưu dựa trên loại dữ liệu đầu vào.

Thách thức So khớp: Hiện tượng Cặp khớp sai (False Matches)

Vấn đề thực tế (Hiệu ứng Spaghetti): Một điểm đặc trưng trong Ảnh truy vấn có thể trông rất giống với nhiều điểm khác nhau trong Ảnh cảnh do hoa văn lặp lại, nhiều nền, hoặc cấu trúc tương đồng.

Hạn chế của Nearest Neighbor (Láng giềng gần nhất): Nếu chỉ chọn điểm có khoảng cách nhỏ nhất một cách mù quáng, máy tính sẽ dễ dàng gán nhầm các cặp khớp mơ hồ. Kết quả là hàng loạt đường chéo lộn xộn, không tập trung vào vật thể thực.

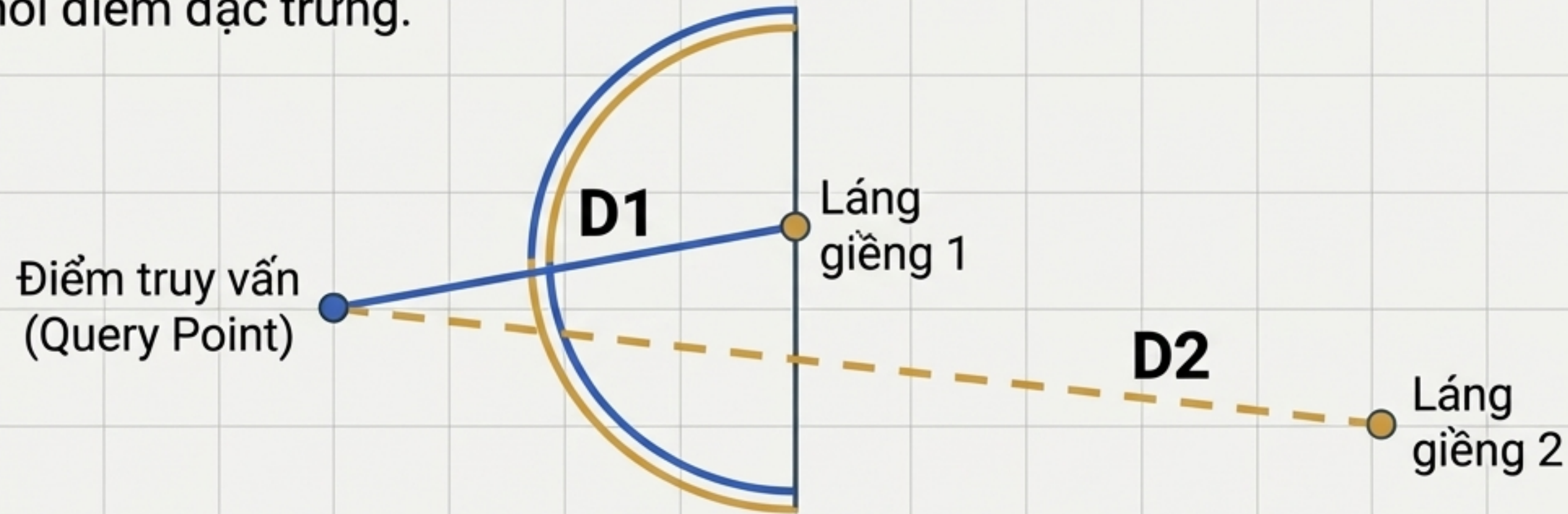
Yêu cầu: Chúng ta cần một màng lọc toán học tinh tế để chỉ giữ lại những cặp khớp thực sự đặc biệt và tin cậy.





Bộ lọc Toán học: Kiểm tra tỷ lệ (Lowe's Ratio Test)

Thay vì chỉ tìm 1 láng giềng, thuật toán dùng KNN để tìm $K = 2$ hàng xóm gần nhất cho mỗi điểm đặc trưng.



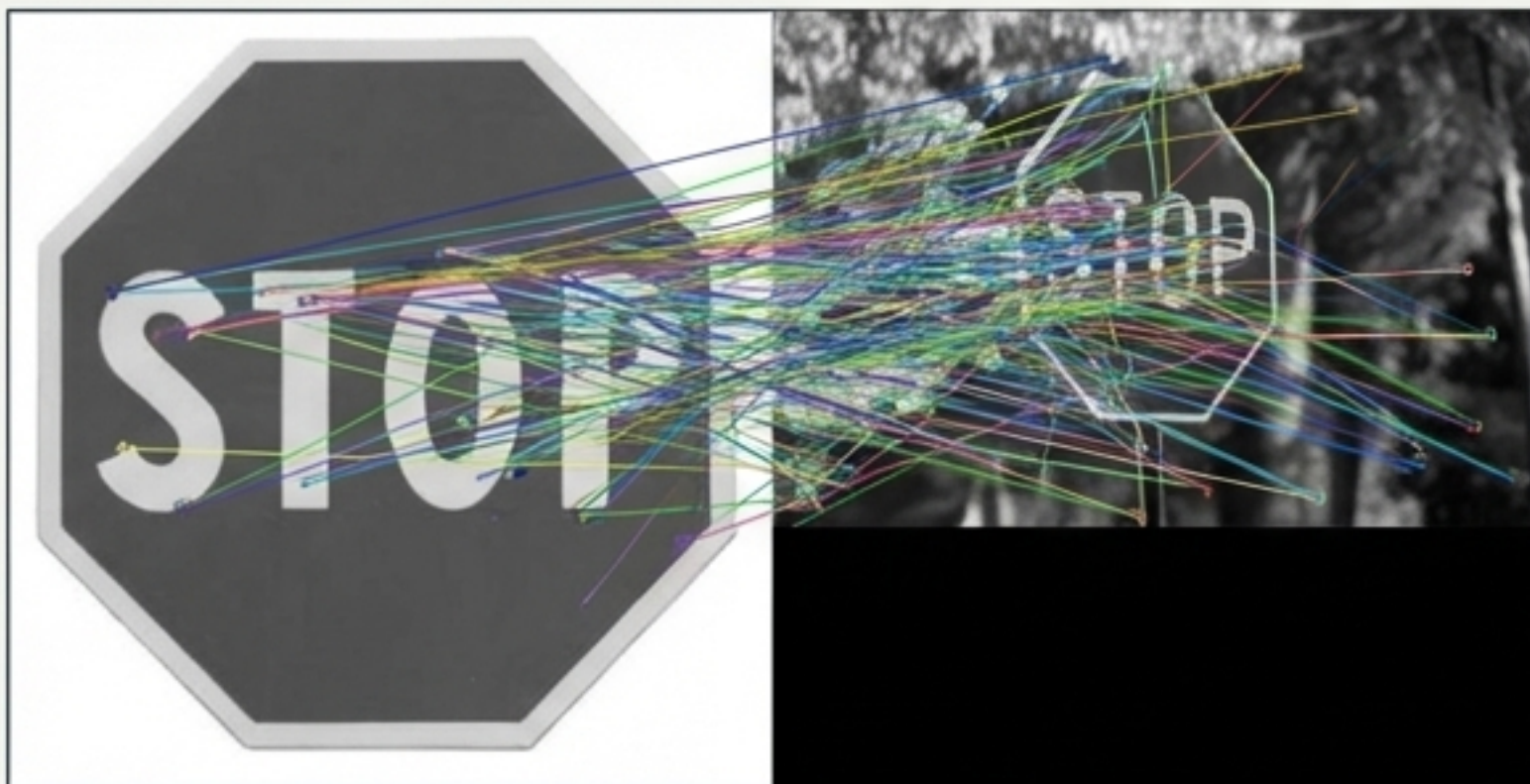
Nguyên lý: Một cặp khớp chỉ được coi là "tốt" nếu điểm gần nhất vượt trội rõ rệt so với điểm gần thứ hai. Nó loại bỏ các điểm có nhiều sự lựa chọn mơ hồ.

Công thức Lọc:

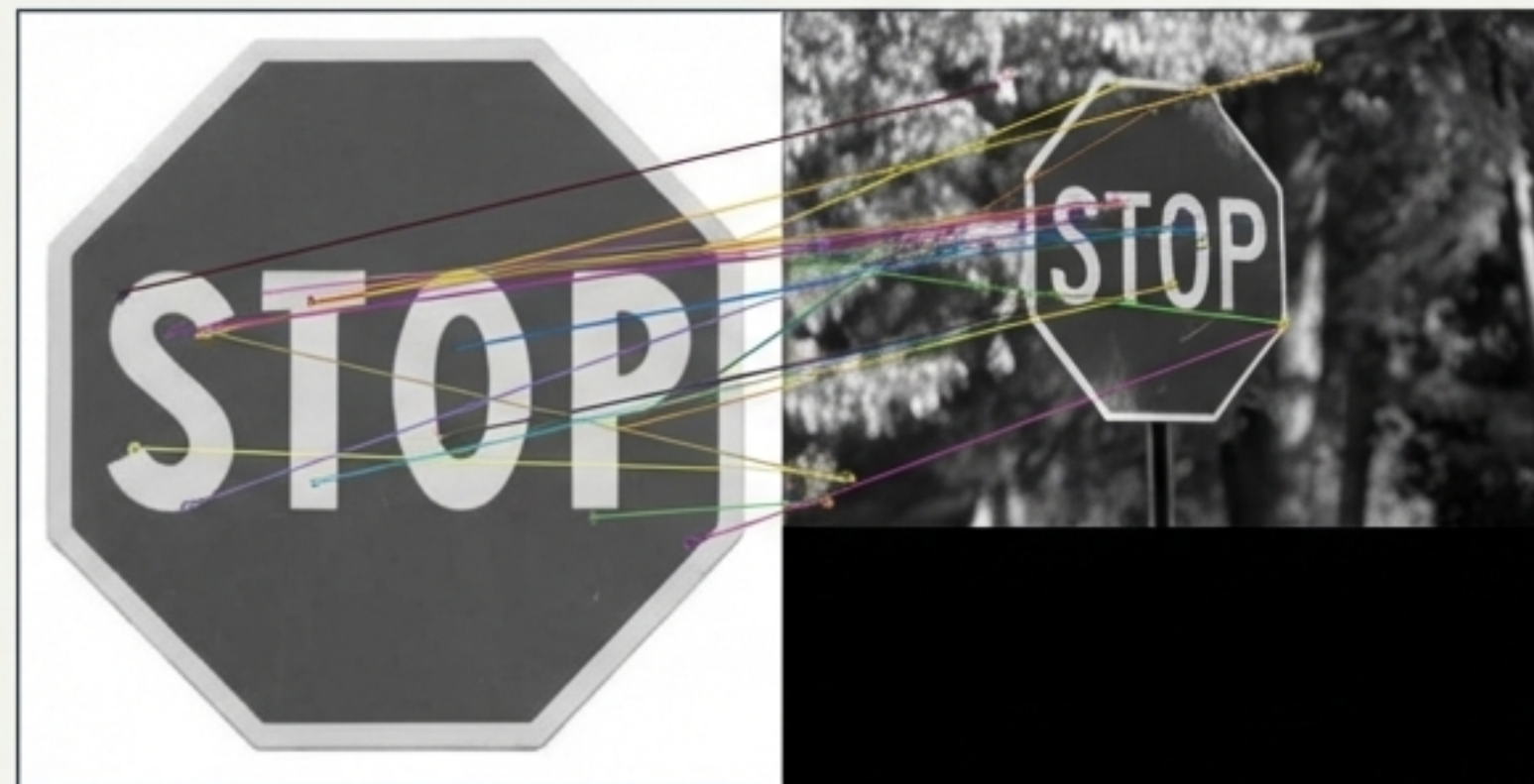
$$D1 < ratio \times D2$$

(Tham số ratio thường được chọn trong khoảng 0.7 đến 0.8)

Sức mạnh của Ratio Test: Trước và Sau



Trước khi lọc: Raw Matches
(Nhiều & Lộn xộn)



Sau khi lọc: Lowe's Ratio Test
(Chính xác & Tập trung)

Kết luận: Ratio Test là bước đệm 'sống còn' để cải thiện độ tin cậy của các cặp khớp trước khi đưa vào các hệ thống hình học phức tạp hơn.



Triển khai với OpenCV: Pipeline 4 Bước

1. Trích xuất (Extract)

```
kp1, des1 = sift.detectAndCompute(img1, None)  
kp2, des2 = sift.detectAndCompute(img2, None)
```

(Trích xuất SIFT keypoints và descriptors)



2. Khởi tạo (Init Matcher)

```
bf = cv2.BFMatcher(cv2.NORM_L2)
```

(Khởi tạo bộ so khớp vết cận với khoảng cách Euclidean)

3. Tìm láng giềng (KNN)

```
matches = bf.knnMatch(des1, des2, k=2)
```

(Tìm 2 ứng viên gần nhất cho mỗi điểm)



4. Lọc Ratio Test (Filter)

```
good_matches = []  
for m, n in matches:  
    if m.distance < 0.75 * n.distance:  
        good_matches.append(m)
```

(Áp dụng công thức Lowe: $D1 < 0.75 \times D2$)

Trực quan hóa Kết quả với drawMatchesKnn

Để kiểm tra độ chính xác, OpenCV cung cấp hàm vẽ các đường kết nối trực tiếp lên không gian ảnh.

```
draw_params = dict(matchColor=(0,255,0),  
                    singlePointColor=None,  
                    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)  
  
result = cv2.drawMatchesKnn(img1, kp1, img2, kp2,  
                             good_matches, None, **draw_params)
```

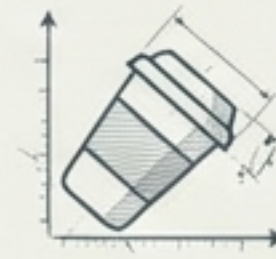
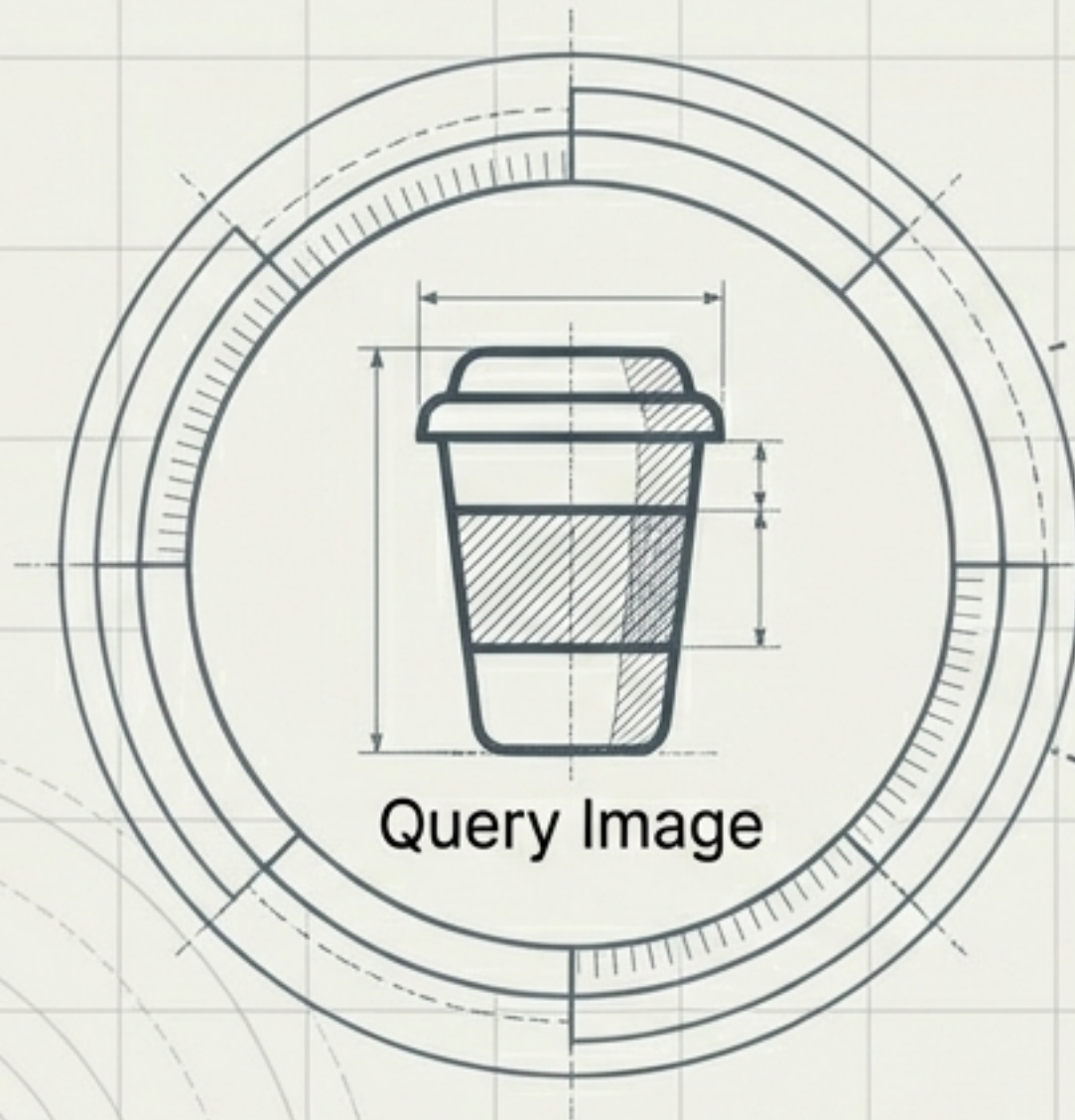
Bí quyết (Pro-tip): Sử dụng cờ NOT_DRAW_SINGLE_POINTS. Cờ này ra lệnh cho máy tính chỉ vẽ những đường nối giữa các cặp khớp thành công, ẩn đi hàng ngàn điểm đặc trưng thừa. Điều này giúp ảnh kết quả sạch sẽ và đối tượng nổi bật rõ rệt.



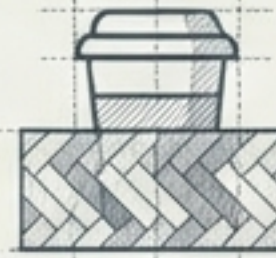


Ứng dụng 1: Nhận dạng và Tìm kiếm Vật thể

Nhờ tính bất biến của các bộ mô tả như SIFT/SURF, hệ thống có thể đối chiếu chính xác và tìm ra vật thể ngay cả trong các điều kiện khắc nghiệt nhất.



1. Thay đổi Tỷ lệ & Góc nhìn (Scale & Rotation): Vật thể bị xoay nghiêng hoặc thu nhỏ ở đẳng xa.



2. Che khuất một phần (Occlusion): Vật thể bị che khuất một phần bởi môi trường.



3. Thay đổi chiếu sáng (Illumination): Bối cảnh tối hoặc chói sáng cực đoan.

Thực tế: Được ứng dụng trong Visual Search (tìm kiếm sản phẩm qua ảnh chụp), nhận diện logo thương hiệu tự động, hoặc giúp robot định vị vật thể cần thao tác trong nhà kho.



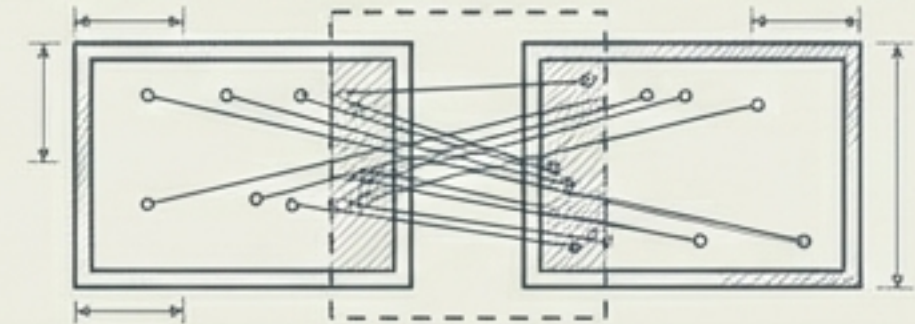
Ứng dụng 2: Xây dựng Ảnh Toàn cảnh (Panorama Stitching)

Ghép các ảnh chụp có phần bối cảnh giao nhau thành một ảnh toàn cảnh liền mạch duy nhất.
So khớp đặc trưng chính là bước "chỉ đường" cho các phép biến đổi hình học.

1

So khớp & Lọc (Matching)

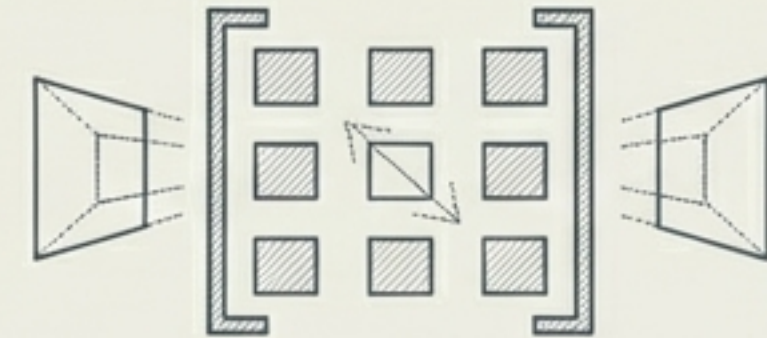
Tìm các cặp điểm tương ứng (Correspondences) giữa 2 bức ảnh (như đã học).



2

Tính Ma trận Homography (Hình học)

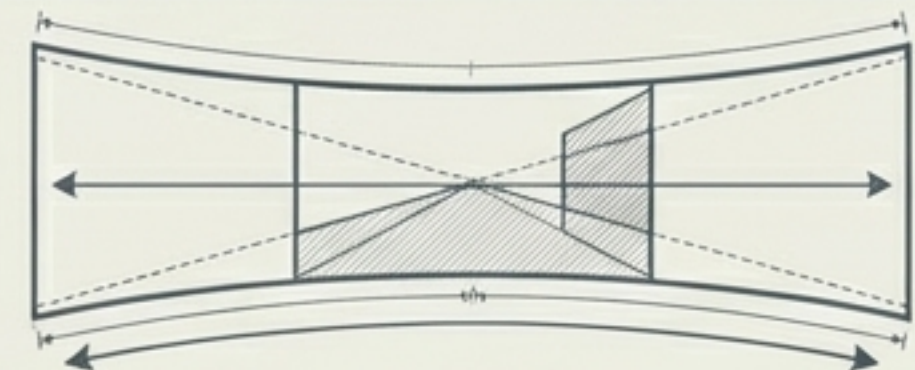
Sử dụng thuật toán RANSAC (thông qua `cv2.findHomography()`) để tính toán phép biến đổi phối cảnh 3D đưa không gian ảnh này về hệ tọa độ của ảnh kia.



3

Làm cong và Ghép (Warping & Stitching)

Sử dụng `cv2.warpPerspective()` để 'uốn cong' bức ảnh theo ma trận Homography và đè chúng lên nhau, tạo ra một trường nhìn (FOV) rộng hơn.





Tổng kết Chương 4 & Bước đệm tương lai

Tiết 1-3: Mức Cơ sở

Khám phá đường biên (Canny), hình học (Hough) và Góc (Harris).
(Máy tính bắt đầu thấy hình dáng)

Tiết 4-7: Mức Mô tả

Làm chủ SIFT, SURF, HOG, LBP.

(Máy tính biến cấu trúc thành vân tay số 128D)

Tiết 8: Mức Liên kết

So khớp (BF/FLANN) và Lọc (Ratio Test).

(Máy tính thiết lập liên kết giữa các thể giới ảnh)

Bước đệm (Chương 5): Phân lớp Ảnh (Image Classification)

Chúng ta đã biết cách trích xuất và so khớp các vector mô tả (Descriptors). Ở chương tiếp theo, chúng ta sẽ dùng chính các vector này làm đầu vào để dạy máy tính tự động phân biệt các loại vật thể (Ví dụ: Chó vs Mèo) bằng thuật toán K-Hàng xóm gần nhất (KNN).